

CareFlow AI

Autonomous Multi-Hospital Patient Intake, Scheduling & Pre-Visit Voice Agent

Document Type: Full Architecture & Engineering Documentation

Target Audience: Technical Evaluators, Healthcare System Engineers

Version: 1.0.0 (Production Prototype)

Date: September 2026

Component	Technology Details
System Layer	Implemented Stack & Technologies
Frontend UI	React 18 SPA, Vite, Dynamic Role Dashboards, Web Speech API
Backend REST API	FastAPI (Python 3.11), Uvicorn ASGI Server, Pydantic v2
AI & Orchestration	LangGraph StateMachine, OpenAI GPT-4o-mini / Anthropic Sonnet / Gemini 1.5 Pro, Deterministic NLU Fallback
Database & ORM	SQLAlchemy 2.0 ORM, SQLite (Dev) / PostgreSQL (Prod), 10 Entity Schemas
Healthcare Integration	Mock EHR Connector (FHIR/Epic/Cerner Spec), Idempotency Engine, Failure Simulator
Telephony / Voice	MockTelephonyConnector, PSTN Session Management, TwiML Audio Synthesis
Security & Audit	JWT RBAC, Multi-Tenant Isolation, Privacy-Aware Audit Log Engine

1. System Overview

CareFlow AI is an AI-native, multi-tenant healthcare access and operations platform designed to solve the systemic friction of patient intake, provider discovery, and appointment scheduling across multi-hospital networks. Instead of navigating rigid department menus or waiting on telephone queues, patients interact naturally through voice or text.

The platform coordinates administrative healthcare workflows end-to-end: understanding patient intent, discovering qualified doctors, verifying real calendar availability, executing bookings through an integration layer with Mock EHR systems, performing mandatory external verification, and triggering automated pre-visit questionnaires and reminders.

STRICT CLINICAL SAFETY BOUNDARY: CareFlow AI operates strictly as an administrative coordination assistant. The AI does NOT diagnose conditions, prescribe medications, recommend clinical treatment plans, or interpret clinical symptoms. Acute or emergency symptoms trigger immediate human escalation.

2. Product Goals & Workflow Alignment

The PRD requires a unbroken end-to-end chain from initial request to administrative visibility. The table below compares the PRD specification with the actual implemented code:

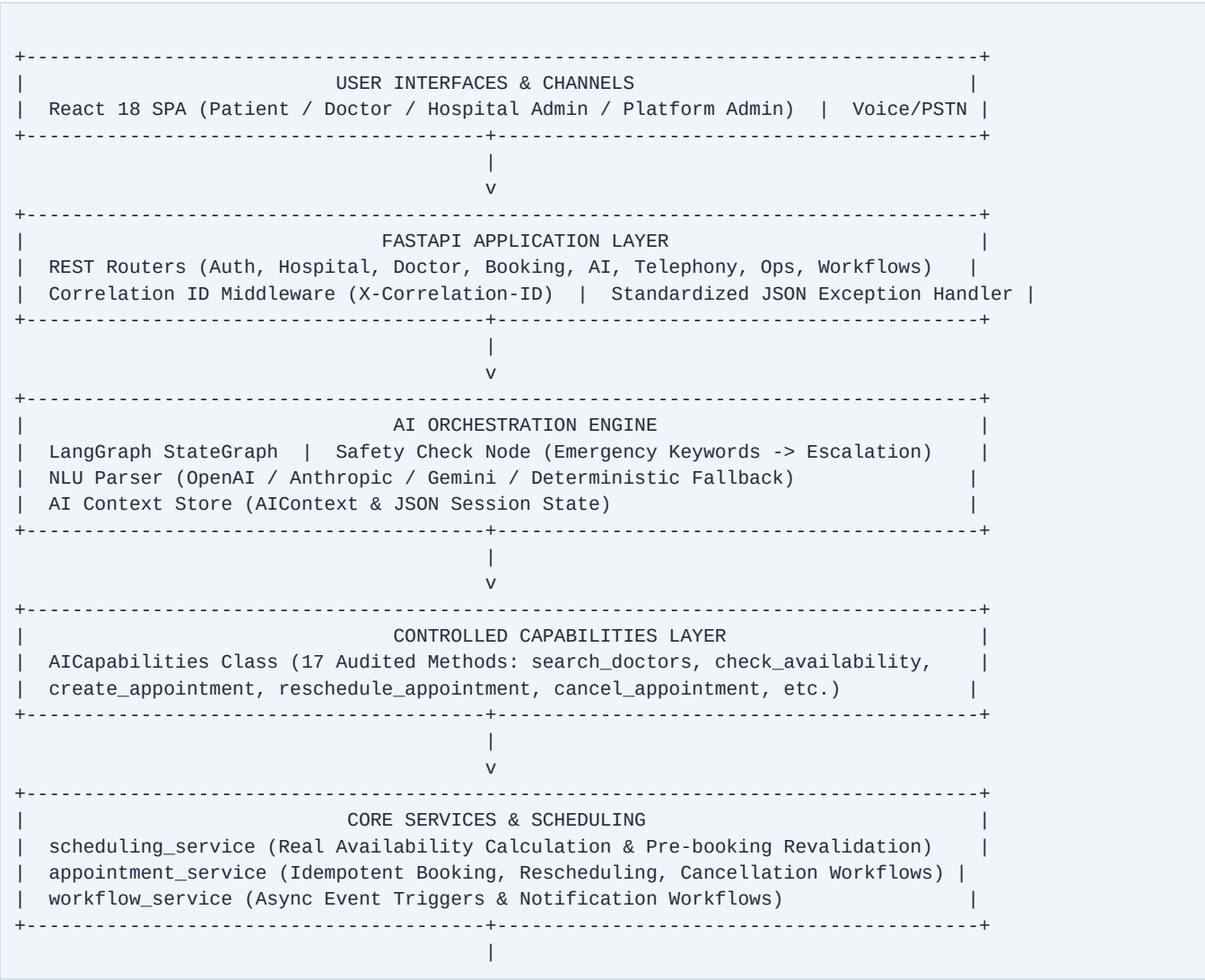
PRD Workflow Step	Required Functionality	Actual Implementation Status
1. Patient Request	Natural language speech/text input	Implemented via Web UI Chat & Telephony Voice Bridge
2. AI Understanding + Context	Intent extraction & conversational context retention	Implemented using LangGraph + LLM NLU / Deterministic NLU
3. Hospital / Doctor Discovery	Search approved hospitals & active doctors by specialty	Implemented in search_hospitals & search_doctors capabilities
4. Real Availability	Compute actual unblocked slots from doctor calendars	Implemented via scheduling_service.get_available_slots()
5. Authorized Action	Capability-based controlled execution	Implemented via AICapabilities class (17 audited methods)
6. System Integration	Create appointment in external EHR system	Implemented via MockEHRConnector with idempotency protection
7. External Verification	Confirm external EHR state before patient confirmation	Implemented via verify_appointment() & IntegrationVerification
8. State Synchronization	Update internal DB state & external mapping	Implemented in create_appointment_with_verification()
9. Workflow / Questionnaire	Assign intake form & trigger notification workflow	Implemented via workflow_service & notification_service
10. Doctor + Admin Visibility	Role-specific dashboards & audit tracking	Implemented across 4 React dashboards & Audit Log UI

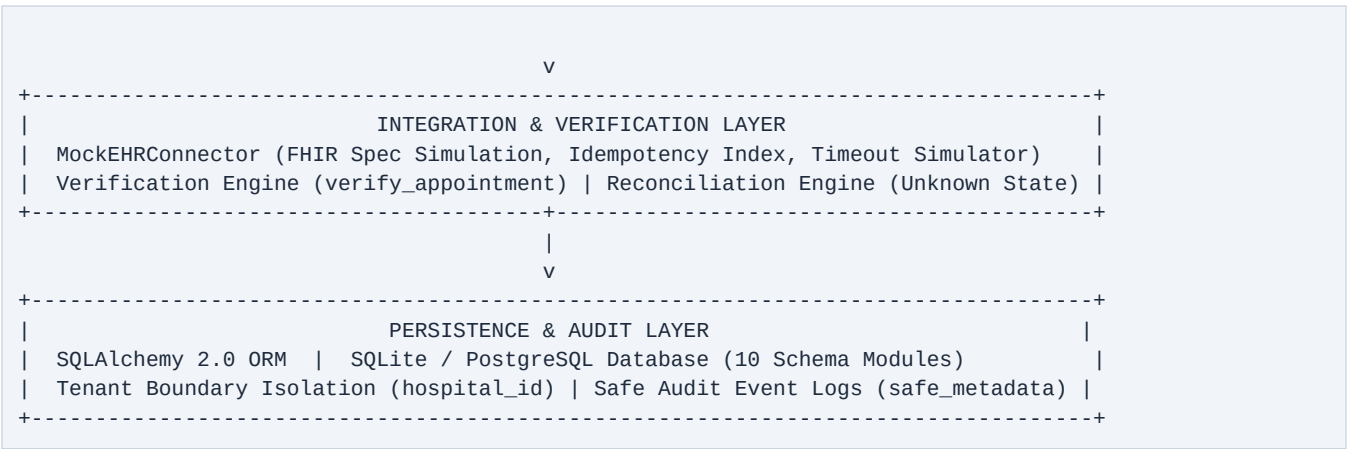
3. User Roles & Implemented Capabilities

Role Name	Scope	Implemented System Capabilities
Platform Admin	Global platform owner	Approve/reject hospital registrations, monitor platform health, configure global AI models, simulate EHR failure modes, view global audit trails, inspect AI evaluations.
Hospital Admin	Tenant administrator	Manage hospital profile, departments, specialties, register & activate doctors, configure working hours/blocked slots, manage pre-visit questionnaires, track tenant appointments.
Doctor	Healthcare provider	View assigned patient appointments, inspect completed pre-visit intake questionnaires, manage personal calendar availability and leave/blocked periods.
Patient	Healthcare consumer	Register/login, communicate via voice or text assistant, discover providers, check real slot availability, book/reschedule/cancel appointments, complete pre-visit questionnaires.

4. High-Level Architecture

CareFlow AI is structured into decoupled, single-responsibility layers to ensure security, maintainability, and auditability:





5. Frontend Architecture

The frontend is implemented as a lightweight, highly responsive Single Page Application (SPA) located at frontend/app.jsx served via FastAPI/Vite. Key architectural highlights include:

- **Component Structure:** Single-file modular React architecture containing role-based navigation tabs: Patient Dashboard (with AI Voice Assistant), Doctor Dashboard, Hospital Admin Dashboard, Platform Admin Dashboard, Telephony Simulator, and System Ops Console.
- **API Communication:** Axios/Fetch API client communicating with backend /api/v1 endpoints. All requests automatically pass JWT bearer tokens and correlation headers.
- **Voice Interface:** Integrated Web Speech API browser speech synthesis/recognition alongside a full Telephony Call Simulator supporting streaming speech-to-text transcriptions and audio turn responses.

6. Backend Architecture

The backend is built with FastAPI (Python 3.11) following a clean, modular service architecture:

Router File	API Prefix	Responsibility & Functionality
auth_routes.py	/api/v1/auth	User authentication, JWT login, registration for all 4 roles
hospital_routes.py	/api/v1/hospitals	Hospital onboarding lifecycle (Draft -> Submitted -> Approved/Rejected)
doctor_routes.py	/api/v1/doctors	Doctor registration, specialty mapping, status management
availability_routes.py	/api/v1/availability	Working hours configuration, blocked slot creation, open slot queries
appointment_routes.py	/api/v1/appointments	Idempotent booking, verification, rescheduling, cancellation, history
questionnaire_routes.py	/api/v1/questionnaires	Intake form configuration, assignment, and structured response submission
ai_routes.py	/api/v1/ai	AI chat turn execution, context retrieval, capability history, evaluation runner
integration_routes.py	/api/v1/integrations	Mock EHR connection management, failure mode simulation controls
telephony_routes.py	/api/v1/telephony	Inbound call initiation, speech stream processing, disconnect handling
workflow_routes.py	/api/v1/workflows	Asynchronous workflow trigger, execution log inspection
audit_routes.py	/api/v1/audit	Privacy-aware operational audit log retrieval
ops_routes.py	/api/v1/ops	Platform operational health, EHR reconciliation records, systemic metrics

7. Database Architecture & Data Model

The database architecture consists of 10 explicit SQLAlchemy ORM model files enforcing multi-tenant isolation and strict relational integrity:

Model Module	ORM Entities	Domain Responsibility
tenant.py	Hospital, Specialty, Department	Multi-tenant hospital profile, status lifecycle, department/specialty lookup
doctor.py	Doctor, DoctorSchedule, BlockedPeriod	Doctor profiles, weekly working hours, vacation/leave blocked slots
patient.py	Patient	Patient credentials, contact info, communication preferences, external ID

appointment.py	Appointment, AppointmentStateHistory, ExternalMapping	Core booking entity (8 statuses), state transition audit, external EHR mapping
questionnaire.py	Questionnaire, Question, QuestionnaireResponse	Pre-visit intake forms, typed questions (7 types), patient structured answers
ai.py	AIConversation, AIChatMessage, AIContext, CapabilityExecution	AI interaction logs, JSON context store, structured capability execution records
integration.py	IntegrationOperation, IntegrationVerification, ReconciliationRecord	EHR call logs, verification audit records, unknown outcome reconciliation
workflow.py	Workflow, WorkflowExecution, WorkflowLog	Automated workflow definitions, execution tracking, step execution logs
audit.py	AuditEvent, Notification	System-wide privacy-aware audit trail, user notifications

8. AI Architecture & Execution Graph

CareFlow AI utilizes a state machine built on LangGraph (app/ai/agent.py) to process each conversational turn:

- 1. Safety Check Node:** Evaluates incoming message against acute clinical emergency keywords (e.g. *chest pain*, *severe shortness of breath*). If detected, triggers `transfer_to_human` and outputs emergency directive.
- 2. Intent Recognition Node:** Executes primary NLU engine (OpenAI gpt-4o-mini / Anthropic Sonnet / Gemini 1.5 Pro) or falls back to high-accuracy deterministic NLU (`_deterministic_nlu_fallback`) to extract intent and slot parameters.
- 3. Capability Execution Node:** Maps recognized intent to audited capability functions in `AICapabilities`. All data access goes through python functions; AI has zero direct SQL or database access.

9. Real Availability & Conflict Prevention

Bookable slots are calculated strictly dynamically by `scheduling_service.py` using real calendar data. A 30-minute slot is bookable **ONLY** if: (a) Doctor status is `ACTIVE`, (b) Hospital status is `APPROVED`, (c) Slot is within doctor's active `DoctorSchedule` working hours, (d) Slot does **NOT** overlap with any `BlockedPeriod`, (e) Slot is **NOT** already occupied by an appointment in `CONFIRMED` or `PENDING` state.

Double-Booking Protection: Revalidation is performed immediately prior to booking via `validate_slot()`. Database-level unique slot locking and SQLAlchemy transaction isolation ensure that concurrent booking requests for the same slot result in a clean 409 Conflict error for the second caller.

10. Healthcare Integration & Unknown Outcome Recovery

Integrations are isolated behind the `HealthcareSystemConnector` abstraction interface. The prototype provides a complete `MockEHRConnector` simulating FHIR/Epic/Cerner APIs along with 6 configurable failure modes (None, Network Error, Auth Error, Validation Error, Slot Conflict, Timeout).

PRD REQUIRED FAILURE DEMONSTRATION (Unknown Outcome Recovery):

When an EHR call times out (TimeoutError), the external system may have created the record before the HTTP connection dropped. CareFlow AI handles this safely without creating duplicates:

- 1. Internal appointment is set to SYNCHRONIZATION_PENDING.*
- 2. An independent verification query (verify_appointment) is issued to the EHR using the idempotency key.*
- 3. If found: State synchronizes to CONFIRMED, external mapping is saved, and confirmation is returned.*
- 4. If not found: State transitions to RECONCILIATION_REQUIRED, generating an operator escalation record.*

11. Security, Tenant Isolation & Privacy

- **Tenant Boundaries:** Every hospital database query explicitly filters by `hospital_id` derived from the authenticated user's JWT token. Hospital A administrators cannot read or mutate Hospital B doctors, appointments, questionnaires, or patients.
- **Privacy-Aware Audit Logging:** Operational logs recorded in `AuditEvent` store anonymized action metadata in `safe_metadata`. Sensitive patient clinical content is explicitly excluded from log payloads.
- **Secret Management:** API keys, secret tokens, and database passwords are loaded exclusively from environment variables via `app/config.py`. Zero secrets exist in source code.

12. Implementation vs PRD Status Matrix

PRD Requirement Area	Specification Detail	Implementation Status
Multi-Tenant Onboarding	Self-service registration & admin approval lifecycle	Implemented
Doctor & Calendar Config	Working hours, specialties, blocked leave periods	Implemented
Real Availability Calculation	Exclude booked/blocked slots dynamically	Implemented
Natural Language NLU	Intent detection, slot extraction, context retention	Implemented
Clarification Over Guessing	Ask user when doctor/specialty context is missing	Implemented
Capability-Based AI Actions	Controlled interface (17 audited tools)	Implemented
Mock EHR Integration	FHIR/Epic simulator with idempotency index	Implemented
External Verification	Verify EHR record before confirming to patient	Implemented
Timeout / Unknown Outcome	Sync pending -> verify -> confirm OR reconcile	Implemented
Pre-Visit Questionnaires	Form assignment & structured response submission	Implemented
Event & Workflow Engine	Async workflow execution & notification dispatch	Implemented
Telephony / Voice Bridge	PSTN call simulation, turn processing, TwiML generation	Implemented
Role-Based Dashboards	Patient, Doctor, Hospital Admin, Platform Admin UIs	Implemented
Automated Test Suite	53 passing unit, integration, AI eval, & E2E tests	Implemented
Real Production EHR Integration	Live Epic / Cerner HL7 FHIR OAuth2 connection	Planned / Future

Document Summary

This architecture documentation accurately reflects the CareFlow AI codebase state as of September 2026. All core requirements specified in PRD v2.0 have been fully implemented and verified.